



Java - Complete course content - 1

1. Introduction to Java

Java is:

- Object-Oriented: Everything in Java is treated as an object.
- Platform-Independent: The Java code is compiled into bytecode, which can run on any platform that has a Java Virtual Machine (JVM).
- Simple and Easy to Learn: It is designed to be easy to use and understand, with a syntax similar to C++.
- Secure: Java provides a secure execution environment that protects against viruses and other security threats.
- Multithreaded: Java has built-in support for multithreading, which allows the execution of multiple tasks concurrently.

2. Simple Java Programs

Here are a few basic examples of Java programs to demonstrate its syntax and structure:

Hello World Program:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!"); // Outputs  
        "Hello, World!"  
    }  
}
```

- Explanation:

- `public class HelloWorld`: Declares a class named HelloWorld.
- `public static void main(String[] args)`: The main method that serves as the entry point for the program.
- `System.out.println()`: Prints a message to the console.

Sum of Two Numbers:

```
import java.util.Scanner;

public class SumNumbers {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter first number: ");
        int num1 = scanner.nextInt();
        System.out.print("Enter second number: ");
        int num2 = scanner.nextInt();

        int sum = num1 + num2;
        System.out.println("The sum is: " + sum);
    }
}
```

- Explanation:
 - `Scanner scanner = new Scanner(System.in)`: Creates a scanner object to take input from the user.
 - `nextInt()`: Reads integer input from the user.
 - The sum of the numbers is calculated and displayed using `System.out.println()`.

3. Applications of Java

Java is used in a wide range of applications, such as:

- **Web Applications**: Java is widely used in building server-side applications with frameworks like Spring, Hibernate, and JavaServer Pages (JSP).

- **Mobile Applications:** Java is the primary language for Android development.
- **Enterprise Solutions:** Java Enterprise Edition (Java EE) is used to develop large-scale enterprise applications.
- **Embedded Systems:** Java can be used for applications running on embedded systems, like smart devices and IoT applications.
- **Desktop GUI Applications:** Java Swing and JavaFX are used to build graphical user interfaces (GUIs).
- **Big Data Technologies:** Java is used in big data processing frameworks like Hadoop and Spark.

4. Structure of a Java Program

A Java program consists of several components, such as classes, methods, and statements. Below is the basic structure of a Java program:

```
// Import statements (optional)
import java.util.Scanner; // Importing the Scanner class

// Class declaration
public class ProgramName {

    // Main method - entry point of the program
    public static void main(String[] args) {

        // Variable declaration and initialization
        int num1 = 5, num2 = 10;

        // Statements and operations
        int sum = num1 + num2;

        // Output the result
        System.out.println("The sum is: " + sum);

    }
}
```

Explanation:

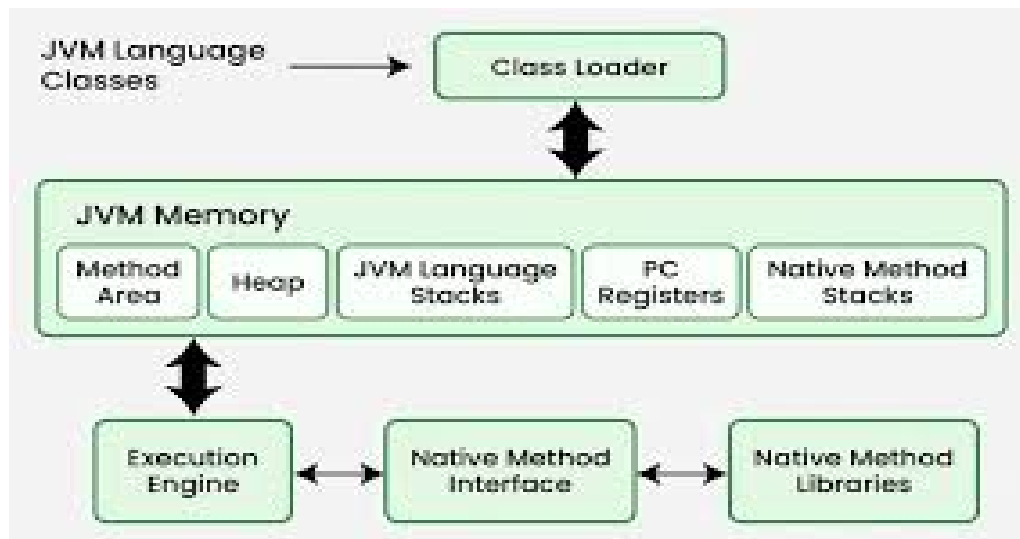
- **Imports:** Used to bring external libraries or classes into the program.

- **Class:** A class is the building block of Java programs. Every Java application must have at least one class with a `main` method.
- **Main Method:** The `main` method is the entry point of any Java program and is executed when the program is run.
- **Statements:** Java statements are used to perform operations, such as variable declarations, calculations, and output.

- **JVM Architecture**

The Java Virtual Machine (JVM) is an integral part of the Java runtime environment that enables Java applications to run on any platform. The JVM provides a layer of abstraction between the compiled Java code (bytecode) and the underlying operating system.

1. **Classloader:** Responsible for loading class files into memory.
2. **Runtime Data Areas:**
 - **Method Area:** Stores class data, method data, and constants.
 - **Heap:** Stores objects and arrays.
 - **Stack:** Stores method calls and local variables.
 - **PC Register:** Keeps track of the current instruction being executed.
 - **Native Method Stack:** Supports native methods (methods written in languages like C/C++).
3. **Execution Engine:** Executes the bytecode instructions.
 - **Interpreter:** Reads bytecode instructions and executes them one by one.
 - **Just-In-Time (JIT) Compiler:** Compiles bytecode into native machine code for faster execution.
4. **Garbage Collector:** Automatically manages memory by reclaiming memory from objects that are no longer in use.



JDK and JRE

- Java Development Kit (JDK):
 - The JDK is a software development kit used to develop Java applications. It includes everything needed to create and run Java programs, such as:
 - JRE (Java Runtime Environment).
 - Compilers: The Java compiler (**javac**) compiles Java source code into bytecode.
 - Debuggers and other tools to aid in development.
- Java Runtime Environment (JRE):
 - The JRE is used to run Java applications. It includes the JVM, core libraries, and other components needed to execute Java programs but does not include the tools for development.
 - JRE provides an environment where Java bytecode can be executed, but it does not provide compilers for creating Java programs.

Comparison:

- JDK is for developers (it includes JRE and tools to develop Java programs).
- JRE is for running Java programs (it includes the JVM but no development tools).

Concept	Explanation
---------	-------------

Java Language	A high-level, object-oriented, platform-independent programming language.
JVM Architecture	A virtual machine that runs Java bytecode and provides platform independence.
JDK	A software development kit that includes JRE, compilers, and development tools.
JRE	A runtime environment that includes JVM and libraries necessary to run Java programs.
Applications of Java	Web applications, mobile apps, enterprise software, desktop apps, etc

Constants, Variables, and Data Types in Java

1. Constants in Java

Constants are variables whose values cannot be changed once they are assigned. In Java, constants are typically defined using the `final` keyword. Once a constant is initialized, it remains constant throughout the program.

Syntax:

```
final <data_type> <variable_name> = <value>;
```

Example:

```
public class ConstantsExample {  
    public static void main(String[] args) {  
        final int MAX_USERS = 100; // MAX_USERS is a constant  
        System.out.println("Max users allowed: " + MAX_USERS);  
    }  
}
```

```
        // Attempting to change the value of a constant will result in
an error
        // MAX_USERS = 150; // This will cause a compile-time error
    }
}
```

- **Key Points:**

- Constants are typically written in uppercase letters to distinguish them from regular variables.
- The value of a constant must be assigned at the time of initialization.
- Once a constant is assigned a value, it cannot be reassigned.

2. Variables in Java

A **variable** in Java is a container that holds data values. Each variable must be declared with a specific **data type**, which defines the kind of data it can store. Variables can be reassigned new values throughout the program.

Types of Variables:

- **Instance Variables:** Declared inside a class but outside any method. They are non-static and are unique to each instance of the class.
- **Class Variables (Static Variables):** Declared with the `static` keyword. They are shared among all instances of the class.
- **Local Variables:** Declared inside a method, constructor, or block. These variables are created when the method is called and destroyed when the method finishes executing.

Syntax for declaring a variable:

```
<data_type> <variable_name> = <initial_value>;
```

Example:

```
public class VariablesExample {
    int instanceVar = 10; // instance variable
    static int classVar = 20; // static variable

    public void myMethod() {
        int localVar = 30; // local variable
        System.out.println("Local variable: " + localVar);
    }
}
```

```

public static void main(String[] args) {
    VariablesExample obj = new VariablesExample();
    obj.myMethod(); // Accessing local variable within method
    System.out.println("Instance variable: " + obj.instanceVar);
    System.out.println("Class variable: " + classVar);
}
}

```

- **Key Points:**

- Instance variables have default values (e.g., `0` for integers, `null` for objects).
- Local variables must be explicitly initialized before use.
- Class variables are shared among all instances of the class.

3. Data Types in Java

In Java, data types specify the type of data a variable can hold. Java has two categories of data types:

1. **Primitive Data Types:** These are the basic types and are predefined by the Java language.
2. **Reference Data Types:** These include objects, arrays, and other types derived from classes.

Primitive Data Types

Java provides the following **primitive data types**:

Data Type	Size	Default Value	Description
<code>byte</code>	1 byte	0	8-bit integer
<code>short</code>	2 bytes	0	16-bit integer
<code>int</code>	4 bytes	0	32-bit integer
<code>long</code>	8 bytes	0L	64-bit integer
<code>float</code>	4 bytes	0.0f	32-bit floating point number
<code>double</code>	8 bytes	0.0d	64-bit floating point number

char	2 bytes	'\u0000'	Single 16-bit Unicode character
boolean	1 byte	false	Represents true/false values

Example of Primitive Data Types:

```
public class PrimitiveDataTypes {
    public static void main(String[] args) {
        int num = 10; // Integer
        double price = 25.50; // Double
        char letter = 'A'; // Character
        boolean isJavaFun = true; // Boolean

        System.out.println("Integer: " + num);
        System.out.println("Double: " + price);
        System.out.println("Character: " + letter);
        System.out.println("Boolean: " + isJavaFun);
    }
}
```

- **Key Points:**

- Primitive data types are directly stored in memory.
- They are more efficient and faster than reference data types.
- The default values are **0** for numeric types, **false** for boolean, and **'\u0000'** for char.

Reference Data Types

Reference data types refer to objects or arrays. These types hold the **memory address** of the object, not the object itself.

- **Classes:** Custom data types that define the properties and behaviors of objects.
- **Arrays:** A collection of similar types of elements.
- **Interfaces:** Define a contract for classes to implement.

Example of Reference Data Types:

```
public class ReferenceDataTypes {
    public static void main(String[] args) {
        String str = "Hello, Java!"; // String (reference type)
        int[] numbers = {1, 2, 3, 4}; // Array (reference type)

        System.out.println("String: " + str);
    }
}
```

```

        System.out.print("Array: ");
        for (int num : numbers) {
            System.out.print(num + " ");
        }
    }
}

```

- **Key Points:**

- Reference data types hold references (memory addresses) to objects, not the actual data.
- Objects and arrays are created dynamically on the heap.
- They are more flexible but consume more memory than primitive types.

Type Casting in Java

Type casting is the process of converting one data type into another. There are two types of casting in Java:

1. **Implicit Casting (Widening):** Automatically converts a smaller type to a larger type.

```

int num = 100;
double result = num; // int to double (automatically done)

```

2. **Explicit Casting (Narrowing):** Manually converting a larger type to a smaller type.

```

double num = 10.99;
int result = (int) num; // double to int (manual casting)

```

Summary of Key Concepts

Concept	Details
Constants	Variables that cannot be reassigned after initialization (use <code>final</code>).
Variables	Containers for data values. Can be instance, static, or local.
Primitive Types	Basic types like <code>int</code> , <code>double</code> , <code>char</code> , <code>boolean</code> , etc.
Reference Types	Types like classes, arrays, and interfaces that hold object references.

Type Casting

Converting data from one type to another, either implicitly or explicitly.

Operators and Expressions in Java

Operators in Java

Java supports a wide range of operators, which can be categorized as follows:

Types of Operators:

1. **Arithmetic Operators:** Used to perform basic arithmetic operations.
 - `+` (Addition)
 - `-` (Subtraction)
 - `*` (Multiplication)
 - `/` (Division)
 - `%` (Modulus - remainder of division)

2. **Relational Operators:** Used to compare two values.
 - `==` (Equal to)
 - `!=` (Not equal to)
 - `>` (Greater than)
 - `<` (Less than)
 - `>=` (Greater than or equal to)
 - `<=` (Less than or equal to)

3. **Logical Operators:** Used to combine conditional statements.
 - `&&` (Logical AND)
 - `||` (Logical OR)
 - `!` (Logical NOT)

4. **Assignment Operators:** Used to assign values to variables.
 - `=` (Simple assignment)
 - `+=`, `-=`, `*=`, `/=`, `%=` (Compound assignment)

5. **Unary Operators:** Operate on a single operand.
 - `++` (Increment)

- `--` (Decrement)
- `+` (Positive)
- `-` (Negative)

6. **Ternary Operator:** A shorthand for if-else statement.

- `condition ? expression1 : expression2`

7. **Bitwise Operators:** Operate on bits and perform bit-by-bit operations.

- `&` (Bitwise AND)
- `|` (Bitwise OR)
- `^` (Bitwise XOR)
- `<<` (Left shift)
- `>>` (Right shift)

Expressions in Java

An **expression** is a combination of values, variables, operators, and method calls that evaluates to a single value.

For example:

```
int sum = 5 + 3; // The expression "5 + 3" evaluates to 8, which is
                // assigned to 'sum'.
```

Other examples:

```
int result = (a + b) * c; // The expression "(a + b) * c" calculates
                          // the result.
boolean isTrue = (x > y) && (y < z); // Logical expression.
```

Decision Making and Branching in Java

Decision making is a key aspect of programming, allowing the program to take different paths depending on certain conditions. Java provides several constructs to handle decision making:

1. **if** Statement

The `if` statement is used to execute a block of code only if a specified condition is `true`.

Syntax:

```
if (condition) {  
    // Block of code to be executed if the condition is true  
}
```

Example:

```
public class IfExample {  
    public static void main(String[] args) {  
        int num = 10;  
  
        if (num > 0) {  
            System.out.println("The number is positive.");  
        }  
    }  
}
```

Explanation:

- The condition `num > 0` is `true`, so the block inside the `if` statement is executed, printing "The number is positive."
-

2. `if-else` Statement

The `if-else` statement allows you to execute one block of code if the condition is `true`, and another block if the condition is `false`.

Syntax:

```
if (condition) {  
    // Block of code to be executed if the condition is true  
} else {  
    // Block of code to be executed if the condition is false  
}
```

Example:

```
public class IfElseExample {
```

```
public static void main(String[] args) {
    int num = -5;

    if (num > 0) {
        System.out.println("The number is positive.");
    } else {
        System.out.println("The number is non-positive.");
    }
}
```

Explanation:

- Since `num` is not greater than 0, the `else` block is executed, printing "The number is non-positive."

3. Nested `if-else` Statement

Nested `if-else` statements are used when there are multiple conditions to be checked in a hierarchical manner.

Syntax:

```
if (condition1) {
    if (condition2) {
        // Block of code if both conditions are true
    } else {
        // Block of code if condition1 is true but condition2 is false
    }
} else {
    // Block of code if condition1 is false
}
```

Example:

```
public class NestedIfExample {
    public static void main(String[] args) {
        int num = 15;

        if (num > 0) {
            if (num % 2 == 0) {
                System.out.println("The number is positive and even.");
            }
        }
    }
}
```

```
        } else {  
            System.out.println("The number is positive and odd.");  
        }  
    } else {  
        System.out.println("The number is non-positive.");  
    }  
}  
}
```

Explanation:

- First, the condition `num > 0` is checked. If `true`, it moves to the inner `if` statement to check whether the number is even or odd.
- For `num = 15`, it prints "The number is positive and odd."

4. switch Statement

The `switch` statement is a more efficient way of handling multiple conditions based on the value of a single variable. It works with primitive types (like `int`, `char`, `byte`, and `short`) and `String` in modern versions of Java.

Syntax:

```
switch (expression) {  
    case value1:  
        // Block of code to be executed if expression == value1  
        break;  
    case value2:  
        // Block of code to be executed if expression == value2  
        break;  
    default:  
        // Block of code to be executed if expression doesn't match any  
case  
}
```

Example:

```
public class SwitchExample {  
    public static void main(String[] args) {  
        int day = 3;  
        switch (day) {  
            case 1:  

```

```

        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 4:
        System.out.println("Thursday");
        break;
    case 5:
        System.out.println("Friday");
        break;
    case 6:
        System.out.println("Saturday");
        break;
    case 7:
        System.out.println("Sunday");
        break;
    default:
        System.out.println("Invalid day");
    }
}
}

```

Explanation:

- The variable `day` is checked against the values inside the `case` statements.
- For `day = 3`, it prints "Wednesday".
- If no `case` matches, the `default` block will be executed.

Statement	Description	Example
if statement	Executes a block of code if the condition is true.	<code>if (num > 0) { System.out.println("Positive"); }</code>
if-else statement	Executes one block of code if the condition is true, and another block if false.	<code>if (num > 0) { System.out.println("Positive"); } else { System.out.println("Negative"); }</code>

nested if-else	Uses multiple if-else statements within each other for multiple conditions.	<pre>if (num > 0) { if (num % 2 == 0) { System.out.println("Even"); } }</pre>
switch statement	Matches a variable with different cases and executes code based on the match.	<pre>switch (day) { case 1: System.out.println("Monday"); break; }</pre>

Looping in Java

In Java, loops are used to execute a block of code repeatedly based on a condition. Loops help avoid repetitive code and make the program more efficient. Java supports several types of loops:

1. Types of Loops in Java

Java provides three main types of loops:

- **For Loop:** It is used when the number of iterations is known before entering the loop.
- **While Loop:** It is used when the number of iterations is not known in advance, and the loop continues as long as a condition is true.
- **Do-while Loop:** Similar to the **while** loop, but it guarantees that the block of code will execute at least once.

1.1. For Loop

The **for loop** is used when the number of iterations is known beforehand. It consists of three parts:

1. Initialization: Initialization of the loop counter.
2. Condition: The condition that must be **true** for the loop to continue.
3. Update: Updates the loop counter after each iteration.

Syntax:

```
for (initialization; condition; update) {
    // Code to be executed
}
```

Example: Print numbers from 1 to 5.

```
public class ForLoopExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i); // Output: 1, 2, 3, 4, 5
        }
    }
}
```

- **Explanation:** The loop runs 5 times, starting from `i = 1` to `i = 5`, printing the value of `i` each time.
-

1.2. While Loop

The **while** loop runs as long as a given condition is true.

Syntax:

```
while (condition) {
    // Code to be executed
}
```

Example: Print numbers from 1 to 5.

```
public class WhileLoopExample {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 5) {
            System.out.println(i); // Output: 1, 2, 3, 4, 5
            i++; // Increment the value of i after each iteration
        }
    }
}
```

- **Explanation:** The loop continues until `i <= 5`. After each iteration, `i` is incremented by 1.
-

1.3. Do-while Loop

The **do-while** loop guarantees that the block of code will execute at least once, even if the condition is **false**.

Syntax:

```
do {  
    // Code to be executed  
} while (condition);
```

Example: Print numbers from 1 to 5.

```
public class DoWhileLoopExample {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            System.out.println(i); // Output: 1, 2, 3, 4, 5  
            i++; // Increment the value of i after each iteration  
        } while (i <= 5);  
    }  
}
```

- **Explanation:** Even though the loop checks the condition after the code block is executed, it ensures that the body of the loop runs at least once.

Classes, Objects, and Methods in Java

Java is an **object-oriented programming (OOP)** language, which means it focuses on objects and classes. Understanding classes, objects, and methods is essential to becoming proficient in Java programming.

1. Classes in Java

A **class** in Java is a blueprint or template for creating objects. It defines the properties (fields) and behaviors (methods) that an object created from the class will have.

Syntax:

```
class ClassName {  
    // Fields (variables)  
    dataType fieldName;  
  
    // Constructor  
    ClassName() {  
        // Initialize object
```

```
}

// Methods
returnType methodName() {
    // Code for method
}
}
```

1.1. Example of a Class in Java

Here is an example of a simple class:

```
public class Car {
    // Fields (properties of the class)
    String color;
    String model;
    int year;

    // Constructor to initialize the object
    public Car(String color, String model, int year) {
        this.color = color;
        this.model = model;
        this.year = year;
    }

    // Method to display car details
    public void displayCarDetails() {
        System.out.println("Car Model: " + model);
        System.out.println("Car Color: " + color);
        System.out.println("Car Year: " + year);
    }
}
```

Explanation:

- The `Car` class has three properties: `color`, `model`, and `year`.
- The constructor `Car(String color, String model, int year)` initializes the object with specific values.
- The `displayCarDetails()` method prints the details of the car.

1.2. Objects in Java

An **object** is an instance of a class. Once a class is defined, objects can be created from that class.

Example:

```
public class Main {  
    public static void main(String[] args) {  
        // Creating an object of the class 'Car'  
        Car myCar = new Car("Red", "Toyota", 2022);  
  
        // Calling the method on the object  
        myCar.displayCarDetails();  
    }  
}
```

- **Explanation:** The object `myCar` is created using the constructor and then calls the method `displayCarDetails()` to print the car details.

1.3. Methods in Java

A **method** is a block of code that performs a specific task and is defined within a class. Methods are used to define the behaviors of an object.

Syntax:

```
returnType methodName(parameters) {  
    // Code to be executed  
}
```

Example of a Method:

```
public class Calculator {  
    // Method to add two numbers  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

In this example, the method `add(int a, int b)` adds two integers and returns the result.

Constructors in Java

A **constructor** is a special method used to initialize objects. It is called when an object of a class is created. A constructor has the same name as the class and does not have a return type.

Types of Constructors:

1. **Default Constructor:** A constructor that does not accept any parameters. If no constructor is defined, Java provides a default constructor.
 2. **Parameterized Constructor:** A constructor that accepts arguments to initialize the object with specific values.
-

2.1. Default Constructor Example

```
public class Person {
    String name;
    int age;

    // Default constructor
    public Person() {
        name = "Unknown";
        age = 0;
    }

    public void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

Explanation:

- When the **Person** object is created without any parameters, the default constructor is used, initializing the **name** and **age** with default values.

2.2. Parameterized Constructor Example

```
public class Person {
    String name;
    int age;

    // Parameterized constructor
```

```
public Person(String name, int age) {
    this.name = name;
    this.age = age;
}

public void displayDetails() {
    System.out.println("Name: " + name);
    System.out.println("Age: " + age);
}
}
```

Explanation:

- This constructor allows us to create a **Person** object with specific values for **name** and **age** provided at the time of creation.

2.3. Using Constructors in Main

```
public class Main {
    public static void main(String[] args) {
        // Using default constructor
        Person person1 = new Person();
        person1.displayDetails();

        // Using parameterized constructor
        Person person2 = new Person("John", 25);
        person2.displayDetails();
    }
}
```

Inheritance in Java

Inheritance is a fundamental concept in Object-Oriented Programming (OOP) that allows one class to inherit the fields and methods of another class. It promotes code reusability and establishes a relationship between classes. In Java, inheritance enables a new class to acquire the properties and behaviors (fields and methods) of an existing class.

1. Basic Concept of Inheritance in Java

In Java, inheritance is implemented using the **extends** keyword. The class that inherits the properties and behaviors is called the **subclass** (or **child class**), and the class from which properties and behaviors are inherited is called the **superclass** (or **parent class**).

Syntax:

```
class SubClass extends SuperClass {  
    // Additional fields and methods of the subclass  
}
```

Example:

```
class Animal {  
    String name;  
  
    // Method of superclass  
    public void eat() {  
        System.out.println(name + " is eating.");  
    }  
}  
  
class Dog extends Animal {  
    // Additional field in subclass  
    String breed;  
  
    // Method of subclass  
    public void bark() {  
        System.out.println(name + " is barking.");  
    }  
}  
  
public class TestInheritance {  
    public static void main(String[] args) {  
        Dog myDog = new Dog();  
        myDog.name = "Buddy";  
        myDog.breed = "Golden Retriever";  
  
        myDog.eat(); // Inherited method  
        myDog.bark(); // Subclass method  
    }  
}
```

Explanation:

- The **Dog** class extends the **Animal** class, meaning it inherits the **eat** method and the **name** field from the **Animal** class.
- The **Dog** class also adds its own method **bark** and an additional field **breed**.

2. Method Overriding

Method Overriding in Java allows a subclass to provide a specific implementation of a method that is already provided by its superclass. The method in the subclass should have the same name, return type, and parameter list as the method in the superclass.

To override a method, we simply define the method with the same signature in the subclass.

Syntax:

```
@Override
public returnType methodName(parameters) {
    // New implementation
}
```

Example:

```
class Animal {
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }
}

public class TestOverriding {
    public static void main(String[] args) {
        Animal myAnimal = new Animal();
        myAnimal.sound(); // Output: Animal makes a sound

        Dog myDog = new Dog();
        myDog.sound(); // Output: Dog barks
    }
}
```

Explanation:

- The `sound()` method is overridden in the `Dog` class. When called on a `Dog` object, it prints "Dog barks", while for an `Animal` object, it prints "Animal makes a sound".
 - The `@Override` annotation is used to indicate that the method in the subclass is overriding a method from the superclass.
-

3. `final` Keyword in Java

The `final` keyword in Java is used to apply restrictions:

- **Final Variables:** A variable declared as `final` cannot be changed after it is initialized. It behaves as a constant.
- **Final Methods:** A method declared as `final` cannot be overridden by subclasses.
- **Final Classes:** A class declared as `final` cannot be extended (no subclass can be created).

3.1. Final Variables

Once a variable is marked `final`, its value cannot be modified after it has been initialized.

Example:

```
class Example {
    final int MAX_VALUE = 100;

    public void changeValue() {
        // MAX_VALUE = 200; // This will cause a compile-time error
    }
}
```

Explanation:

- The `MAX_VALUE` variable cannot be modified once it is assigned a value.

3.2. Final Methods

A method marked as `final` cannot be overridden in any subclass.

Example:

```
class Animal {
    public final void makeSound() {
        System.out.println("Animal makes a sound");
    }
}
```

```
    }  
}  
  
class Dog extends Animal {  
    // This would cause an error because makeSound() is final in the  
    // superclass  
    // public void makeSound() {  
    //     System.out.println("Dog barks");  
    // }  
}
```

Explanation:

- The `makeSound()` method in the `Animal` class is marked as `final`, meaning it cannot be overridden in the `Dog` class.

3.3. Final Classes

A class marked as `final` cannot be subclassed.

Example:

```
final class Animal {  
    // Class code here  
}  
  
// The following will cause an error:  
class Dog extends Animal { // Compile-time error: Cannot subclass final  
    class Animal  
}
```

Explanation:

- The `Animal` class is declared `final`, so no class can extend it.

4. The `finalize()` Method in Java

The `finalize()` method is used to perform cleanup activities before an object is destroyed. It is invoked by the Garbage Collector (GC) just before the object is removed from memory.

However, the use of `finalize()` is not recommended in modern Java, as it is unreliable and does not guarantee that the method will always be executed before object destruction. Instead, developers should use `try-with-resources` or explicit cleanup methods.

Syntax:

```
protected void finalize() throws Throwable {  
    // Code to clean up resources  
    super.finalize();  
}
```

Example:

```
class MyClass {  
    protected void finalize() {  
        System.out.println("Object is being garbage collected.");  
    }  
  
    public static void main(String[] args) {  
        MyClass obj = new MyClass();  
        obj = null; // Eligible for garbage collection  
  
        System.gc(); // Request garbage collection  
    }  
}
```

Explanation:

- When `obj` is set to `null`, it becomes eligible for garbage collection. The `finalize()` method will be invoked just before the object is collected, printing "Object is being garbage collected."
- `System.gc()` is a request for garbage collection, but it is not guaranteed that it will run immediately.

5. Abstract Classes and Abstract Methods

An **abstract class** in Java is a class that cannot be instantiated directly. It is meant to be subclassed by other classes. Abstract classes can have both abstract methods (methods

without a body) and non-abstract methods (methods with a body). The abstract methods must be implemented by the subclasses.

- **Abstract Method:** A method declared without a body and with the `abstract` keyword. It must be implemented by subclasses.

5.1. Abstract Class Example

```
abstract class Animal {
    // Abstract method (does not have a body)
    public abstract void sound();

    // Regular method
    public void eat() {
        System.out.println("Animal is eating.");
    }
}

class Dog extends Animal {
    // Providing implementation for the abstract method
    public void sound() {
        System.out.println("Dog barks");
    }
}

public class TestAbstract {
    public static void main(String[] args) {
        // Animal a = new Animal(); // This would cause an error
        // because Animal is abstract
        Dog myDog = new Dog();
        myDog.sound(); // Output: Dog barks
        myDog.eat();   // Output: Animal is eating
    }
}
```

Explanation:

- The class `Animal` is abstract, and it has an abstract method `sound()`. The subclass `Dog` provides the implementation of this method.
- You cannot create an instance of an abstract class directly, but you can create instances of its concrete subclasses.

Concept	Description
---------	-------------

Inheritance	A mechanism where a new class inherits properties and methods from an existing class.
Method Overriding	A subclass provides a specific implementation of a method that is already defined in the superclass.
final Keyword	Used to define constants (variables), prevent method overriding, and prevent class inheritance.
finalize() Method	A method that is invoked by the garbage collector before an object is destroyed, typically used for cleanup.
Abstract Classes	Classes that cannot be instantiated and may contain abstract methods that must be implemented by subclasses.
Abstract Methods	Methods in an abstract class that do not have a body and must be implemented by subclasses

Arrays in Java

An **array** in Java is a container object that holds a fixed number of values of a single data type. Arrays are used to store data in a structured manner and allow fast access to elements using indices.

1.1. Characteristics of Arrays

- **Fixed Size:** Once an array is created, its size cannot be changed. You must specify the size of the array when you create it, and the array can only hold that many elements.
- **Indexed:** Arrays are indexed, starting from 0. This means that the first element is accessed at index 0, the second at index 1, and so on.
- **Homogeneous:** All elements in an array must be of the same type, either primitive data types or objects.

1.2. Declaring and Initializing Arrays

There are two ways to declare and initialize arrays in Java:

1. Array Declaration and Initialization at the Same Time:

Syntax:

```
dataType[] arrayName = {value1, value2, value3, ...};
```

Example:

```
int[] numbers = {10, 20, 30, 40, 50};
```

2. Array Declaration and Initialization Separately:

Syntax:

```
dataType[] arrayName = new dataType[size];
```

Example:

```
int[] numbers = new int[5]; // Creates an array of 5 integers
```

1.3. Accessing Array Elements

Array elements are accessed using the index, with indices starting from 0. You can also modify elements in the array.

Example:

```
int[] numbers = {10, 20, 30, 40, 50};  
System.out.println(numbers[0]); // Output: 10  
numbers[2] = 35; // Modifying the third element  
System.out.println(numbers[2]); // Output: 35
```

1.4. Array Length

In Java, the **length** of an array is accessed using the **length** property (not a method). The **length** property tells you how many elements an array can hold.

Example:

```
int[] numbers = {10, 20, 30, 40, 50};  
System.out.println("Length of array: " + numbers.length); // Output: 5
```

1.5. Multi-Dimensional Arrays

Java supports arrays with more than one dimension. The most common multi-dimensional arrays are **2-D arrays**, which are often used to represent tables, grids, or matrices.

Example of 2-D Array:

```
int[][] matrix = {
```

```
{1, 2, 3},  
{4, 5, 6},  
{7, 8, 9}  
};  
System.out.println(matrix[0][1]); // Output: 2
```

In the example above, `matrix[0][1]` refers to the element in the first row and second column of the 2-D array.

Vectors in Java

A **Vector** is a dynamic array that grows or shrinks automatically when elements are added or removed. It is part of the **Java Collections Framework** and is found in the `java.util` package. Vectors are similar to arrays but offer more flexibility in managing the size of the collection.

2.1. Characteristics of Vectors

- **Dynamic Size:** Unlike arrays, vectors can automatically grow or shrink in size. When the number of elements exceeds the vector's capacity, it will automatically increase the capacity.
- **Resizable:** Vectors start with a default capacity (usually 10 elements), and if more elements are added, the vector's size grows. You can also set the capacity manually.
- **Homogeneous:** Similar to arrays, vectors can only store elements of the same data type (objects). However, since vectors are part of the Collections Framework, you typically store objects (instead of primitives) in a vector.

2.2. Declaring and Creating Vectors

To create a vector, you use the `Vector` class, which is part of the `java.util` package.

Syntax:

```
Vector<dataType> vectorName = new Vector<>();
```

You can also specify the initial capacity and the **capacity increment** (the amount by which the vector's size increases when it is full).

Example:

```
import java.util.Vector;
```

```

public class VectorExample {
    public static void main(String[] args) {
        // Create a Vector with initial capacity of 3
        Vector<Integer> vector = new Vector<>(3);

        // Adding elements to the Vector
        vector.add(10);
        vector.add(20);
        vector.add(30);

        // Print Vector
        System.out.println("Vector Elements: " + vector);
    }
}

```

2.3. Common Vector Operations

- **Adding Elements:** You can add elements to a vector using the `add()` method.

```
vector.add(40);
```

- **Accessing Elements:** You can access elements using the `get()` method and by specifying an index.

```
Integer element = vector.get(2); // Gets the element at index 2
```

- **Removing Elements:** You can remove elements from a vector using the `remove()` method or `removeElement()` method.

```
vector.remove(1); // Removes element at index 1
```

- **Size of Vector:** The `size()` method returns the number of elements in the vector.

```
int size = vector.size(); // Returns the number of elements in the vector
```

- **Capacity of Vector:** The `capacity()` method returns the current capacity of the vector (i.e., the total number of elements it can hold before it needs to resize).

```
int capacity = vector.capacity(); // Returns the capacity of the vector
```

2.4. Vector vs Array

Here's a comparison between arrays and vectors:

Feature	Array	Vector
---------	-------	--------

Size	Fixed size once created.	Dynamic size, grows as elements are added.
Memory	Memory is allocated at creation time.	Memory is allocated dynamically as elements are added.
Access Type	Direct access via index.	Direct access via index, but involves more overhead due to dynamic resizing.
Type of Data Stored	Can store primitives or objects.	Can store only objects (no primitive types).
Performance	Faster as there is no resizing.	Slower due to resizing and synchronization overhead.
Resizing	Cannot be resized once created.	Automatically resizes when capacity is exceeded.

Summary

Arrays:

- **Fixed Size:** The size of an array is specified when the array is created and cannot be changed.
- **Homogeneous Elements:** All elements must be of the same data type.
- **Efficient:** Direct access to elements is faster as the array size is fixed.
- **Indexing:** Arrays use integer indices starting from 0.
- **Types:** Can hold primitive data types or objects.

Vectors:

- **Dynamic Size:** Vectors can automatically resize as elements are added.
- **Resizable:** When the number of elements exceeds the vector's capacity, it automatically expands.
- **Stores Objects:** Vectors can store only objects (no primitive types).
- **Slower:** Vectors tend to be slower than arrays due to dynamic resizing and synchronization.
- **Part of Collections Framework:** Vectors are part of the Java Collections Framework and implement the `List` interface.

Strings in Java

In Java, a **String** is a sequence of characters that is widely used for representing text. Strings in Java are objects that belong to the `String` class, which is part of the `java.lang` package. Strings in Java are **immutable**, meaning once a String object is created, its value cannot be changed. Instead of modifying the original string, any operation on a string creates a new string.

1. String Declaration and Initialization

In Java, there are two primary ways to declare and initialize a string:

Using String Literals: When a string is declared using double quotes (""), Java automatically stores it in a special pool of strings known as the **string pool** (or string literal pool). This allows efficient memory usage as identical string literals are shared across the program.

Example:

java

```
String str = "Hello, World!";
```

1.

Using the new Keyword: You can also create a string object using the **new** keyword. This approach does not utilize the string pool, meaning a new string object is created in memory.

Example:

java

```
String str = new String("Hello, World!");
```

2.

2. Immutable Nature of Strings

Strings in Java are **immutable**, meaning that once a String object is created, its value cannot be changed. If you perform any modification (e.g., concatenation, replacing characters, etc.), a new string object is created.

Example:

java

```
String str = "Hello";  
str = str + " World"; // A new string object is created  
System.out.println(str); // Output: Hello World
```

In the above example, the original string "Hello" remains unchanged, and a new string "Hello World" is created and assigned to the variable **str**.

3. String Operations

Java provides a wide range of methods to manipulate strings. Here are some of the most common string operations:

3.1. Length of a String

You can get the length of a string using the `length()` method. It returns the number of characters in the string.

Example:

java

```
String str = "Hello, World!";  
System.out.println(str.length()); // Output: 13
```

3.2. Accessing Characters in a String

To access a specific character in a string, you can use the `charAt()` method. It returns the character at a given index (starting from 0).

Example:

java

```
String str = "Hello";  
char ch = str.charAt(1); // Accessing the character at index 1  
System.out.println(ch); // Output: e
```

3.3. String Concatenation

You can concatenate (combine) two strings using the `+` operator or the `concat()` method.

Using the `+` operator:

java

```
String str1 = "Hello";  
String str2 = " World!";  
String result = str1 + str2;  
System.out.println(result); // Output: Hello World!
```

-

Using the `concat()` method:

java

```
String str1 = "Hello";
String str2 = " World!";
String result = str1.concat(str2);
System.out.println(result); // Output: Hello World!
```

-

3.4. Substring

To extract a part of the string, use the `substring()` method. It takes either one or two parameters: the start index (inclusive) and the end index (exclusive).

Example (extracting from a specific index):

java

```
String str1 = "Hello";
String str2 = " World!";
String result = str1.concat(str2);
System.out.println(result); // Output: Hello World!
```

-

Example (extracting between two indices):

java

```
String str = "Hello, World!";
String substr = str.substring(0, 5); // Extracts from index 0 to 4
System.out.println(substr); // Output: Hello
```

-

3.5. String Comparison

To compare strings, use the `equals()` method for content comparison, and `compareTo()` for lexicographical comparison.

Using `equals()`:

java

```
String str1 = "Hello";
String str2 = "Hello";
String str3 = "World";
```

```
System.out.println(str1.equals(str2)); // Output: true
System.out.println(str1.equals(str3)); // Output: false
```

-

Using **compareTo()**: The **compareTo()** method returns a value less than 0 if the first string is lexicographically smaller, greater than 0 if larger, and 0 if they are equal.

java

```
String str1 = "apple";
String str2 = "banana";
System.out.println(str1.compareTo(str2)); // Output: -1
```

-

3.6. Converting Case

You can convert the case of a string using the following methods:

- **toUpperCase()** converts the string to uppercase.
- **toLowerCase()** converts the string to lowercase.

Example:

java

```
String str = "Hello, World!";
System.out.println(str.toUpperCase()); // Output: HELLO, WORLD!
System.out.println(str.toLowerCase()); // Output: hello, world!
```

3.7. Trimming Whitespaces

The **trim()** method removes any leading and trailing whitespace from the string.

Example:

java

```
String str = " Hello, World! ";
System.out.println(str.trim()); // Output: Hello, World!
```

4. Special String Methods

Java provides several specialized methods to manipulate strings:

4.1. `contains()`

The `contains()` method checks if a particular substring exists within the string.

Example:

java

```
String str = "Hello, World!";  
System.out.println(str.contains("World")); // Output: true
```

4.2. `replace()`

The `replace()` method replaces occurrences of a character or substring with another character or substring.

Example:

```
String str = "Hello, World!";  
String newStr = str.replace("World", "Java");  
System.out.println(newStr); // Output: Hello, Java!
```

4.3. `split()`

The `split()` method splits a string into an array of substrings based on a specified delimiter.

Example:

```
String str = "apple,banana,cherry";  
String[] fruits = str.split(",");  
for (String fruit : fruits) {  
    System.out.println(fruit);  
}  
// Output:  
// apple  
// banana  
// cherry
```

4.4. `indexOf()`

The `indexOf()` method returns the index of the first occurrence of a substring or character.

Example:

```
String str = "Hello, World!";
int index = str.indexOf("World");
System.out.println(index); // Output: 7
```

4.5. `startsWith()` and `endsWith()`

These methods check whether a string starts or ends with a given substring.

Example:

```
String str = "Hello, World!";
System.out.println(str.startsWith("Hello")); // Output: true
System.out.println(str.endsWith("!")); // Output: true
```

StringBuffer vs StringBuilder

In addition to the `String` class, Java also provides two other classes: `StringBuffer` and `StringBuilder`, both of which are mutable (can be changed). These are used when you expect frequent modifications to the string.

- **StringBuffer**: Thread-safe, meaning it can be used in a multi-threaded environment but at the cost of performance due to synchronization.
- **StringBuilder**: Not thread-safe, but faster than `StringBuffer` in single-threaded applications.

Example of StringBuilder:

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(", World!");
System.out.println(sb.toString()); // Output: Hello, World!
```

Summary of Important String Methods

Method	Description	Example
<code>length()</code>	Returns the length of the string.	<code>str.length()</code>

<code>charAt(int index)</code>	Returns the character at the specified index.	<code>str.charAt(0)</code>
<code>substring(int start)</code>	Returns a substring from the start index to the end.	<code>str.substring(7)</code>
<code>substring(int start, int end)</code>	Returns a substring between the specified indices.	<code>str.substring(0, 5)</code>
<code>equals(Object obj)</code>	Checks if two strings are equal in content.	<code>str.equals("Hello")</code>
<code>equalsIgnoreCase(String other)</code>	Compares two strings ignoring case.	<code>str.equalsIgnoreCase("HELLO")</code>
<code>toUpperCase()</code>	Converts the string to uppercase.	<code>str.toUpperCase()</code>
<code>toLowerCase()</code>	Converts the string to lowercase.	<code>str.toLowerCase()</code>
<code>replace(CharSequence oldChar, CharSequence newChar)</code>	Replaces a substring with another.	

(Remaining topics covered in Part-2)